Autoconf 자동 설정

개요

Autoconf는 IPFS 네트워크 설정을 자동으로 가져오고 캐싱하는 클라이언트 라이브러리입니다. 이 모듈은 HTTP를 통해 네트워크 설정 파일(autoconf.json)을 가져와서 로컬에 캐시하고, 백그라운드에서 주기적으로 업데이트합니다. 네트워크 연결이 불안정한 상황에서도 캐시된 설정을 사용하여 안정적으로 동작할 수 있습니다.

주요 개념

AutoConf 설정 파일

AutoConf 설정 파일은 IPFS 네트워크의 다음과 같은 정보를 포함합니다:

- **SystemRegistry**: 알려진 라우팅 시스템들과 그들의 기능 목록
- **DelegatedEndpoints**: 특정 라우팅 작업을 구현하는 HTTP 엔드포인트들
- **DNSResolvers**: .eth 및 기타 도메인을 위한 DNS-over-HTTPS 리졸버들

캐싱 시스템

- **버전 기반 캐싱**: 각 설정 버전이 별도 파일로 저장됨
- **HTTP 조건부 요청**: ETag/Last-Modified를 사용한 효율적인 업데이트
- **자동 정리**: 오래된 버전들이 자동으로 삭제됨 (기본값: 3개 버전 유지)
- **호스트명 기반 분리**: 여러 URL의 설정이 독립적으로 캐시됨

기본 사용법

간단한 클라이언트 생성

```go
package main

import (
"context"
"fmt"
"log"
"github.com/ipfs/boxo/autoconf"
)

func main() {
// 기본 설정으로 클라이언트 생성 (메인넷 URL과 폴백 사용)
client, err := autoconf.NewClient()
if err != nil {
log.Fatal(err)
}
// 캐시 우선 접근: 캐시된 설정 사용, 네트워크 요청 없음
config := client.GetCached()
// 설정 데이터 사용
nativelySupported := []string{"AminoDHT"} // 로컬에서 실행하는 시스템들
bootstrapPeers := config.GetBootstrapPeers(nativelySupported...)
delegatedEndpoints := config.GetDelegatedEndpoints(nativelySupported...)
```

```
dnsResolvers := config.GetDNSResolvers()

fmt.Printf("부트스트랩 피어: %v\n", bootstrapPeers)

fmt.Printf("위임된 엔드포인트: %v\n", delegatedEndpoints)

fmt.Printf("DNS 리졸버: %v\n", dnsResolvers)

}
```

새로고침과 함께 사용

```
func main() {

client, err := autoconf.NewClient()

if err != nil {

log.Fatal(err)

}

// 새로고침 시도: 네트워크 업데이트 시도, 실패 시 캐시나 기본값 사용

config := client.GetCachedOrRefresh(context.Background())

// 설정 접근 방법은 동일 - 항상 nil이 아닌 값 보장

nativelySupported := []string{"AminoDHT"}

bootstrapPeers := config.GetBootstrapPeers(nativelySupported...)

fmt.Printf("부트스트랩 피어: %v\n", bootstrapPeers)

}
```

고급 활용

사용자 정의 옵션으로 클라이언트 생성

```go
func  createCustomClient() (*autoconf.Client, error) {
return  autoconf.NewClient(
// 기본 설정
autoconf.WithURL("https://conf.ipfs-mainnet.org/autoconf.json"),
autoconf.WithRefreshInterval(24*time.Hour),
autoconf.WithCacheDir("/path/to/cache"),
autoconf.WithUserAgent("my-app/1.0"),
autoconf.WithCacheSize(autoconf.DefaultCacheSize),
autoconf.WithTimeout(autoconf.DefaultTimeout),
autoconf.WithFallback(autoconf.GetMainnetFallbackConfig),
// 백그라운드 새로고침을 위한 선택적 콜백들
autoconf.WithOnNewVersion(func(oldVer, newVer  int64, url  string) {
log.Printf("설정이 v%d에서 v%d로 업데이트됨 (URL: %s)", oldVer, newVer, url)
// 애플리케이션 재시작이나 설정 다시 로드 트리거
}),
autoconf.WithOnRefresh(func(resp *autoconf.Response) {
log.Printf("설정 새로고침됨: v%s at %s", resp.Version, resp.FetchTime)
// 메타데이터 저장이나 종속 작업 트리거 가능
}),
autoconf.WithOnRefreshError(func(err  error) {
log.Printf("새로고침 실패: %v", err)
```

```
// 알림 전송이나 백업 엔드포인트로 전환 가능
}),
)
}
```

명시적 오류 처리

```
func explicitErrorHandling(ctx context.Context) error {
client, err := autoconf.NewClient()
if err != nil {
return err
}
// GetLatest는 명시적 처리를 위해 오류를 반환
response, err := client.GetLatest(ctx)
if err != nil {
return fmt.Errorf("설정 가져오기 실패: %w", err)
}
// 설정과 메타데이터에 접근
config := response.Config
fmt.Printf("설정: %+v\n", config)
fmt.Printf("가져온 시간: %v\n", response.FetchTime)
fmt.Printf("버전: %s\n", response.Version)
fmt.Printf("캐시에서 가져옴: %v\n", response.FromCache())
```

```
    fmt.Printf("캐시 나이: %v\n", response.CacheAge)

    return  nil

}
```

설정 옵션

기본 설정 옵션

- **WithURL(url)**: autoconf URL 추가 (로드 밸런싱을 위해 여러 번 호출 가능)
- **WithRefreshInterval(duration)**: autoconf 데이터 새로고침 간격 설정 (기본값: 24시간)
- **WithCacheDir(dir)**: 사용자 정의 캐시 디렉토리 설정
- **WithCacheSize(n)**: 최대 캐시된 버전 수 설정 (기본값: 3)
- **WithFallback(func)**: 가져오기 실패 시 설정을 반환하는 폴백 함수 설정

HTTP 설정 옵션

- **WithHTTPClient(client)**: 사용자 정의 HTTP 클라이언트 사용
- **WithUserAgent(ua)**: HTTP 사용자 에이전트 설정
- **WithTimeout(duration)**: HTTP 타임아웃 설정 (기본값: 5초)
- **WithTLSInsecureSkipVerify(bool)**: TLS 검증 건너뛰기 (테스트용)

백그라운드 새로고침 콜백

- **WithOnNewVersion(func)**: 새로운 설정 버전 감지 시 호출
- **WithOnRefresh(func)**: 성공적인 새로고침 후 호출
- **WithOnRefreshError(func)**: 새로고침 실패 시 호출

실제 예제

애플리케이션 통합 예제

```go
package main

import (
"context"
"log"
"os"
"os/signal"
"syscall"
"time"
"github.com/ipfs/boxo/autoconf"
)

func main() {
// 사용자 정의 설정으로 클라이언트 생성
client, err := autoconf.NewClient(
autoconf.WithCacheDir("/app/data/autoconf"),
autoconf.WithUserAgent("myapp/1.2.3"),
autoconf.WithTimeout(autoconf.DefaultTimeout),
autoconf.WithOnNewVersion(func(oldVer, newVer int64, url string) {
```

```
log.Printf("새 설정 버전 %d 사용 가능", newVer)
})
)
if err != nil {
log.Fatal(err)
}
// 턴키 솔루션 사용 - 실패하지 않고 항상 사용 가능한 설정 반환
config := client.GetCachedOrRefresh(context.Background())
// 모든 설정 타입에 접근
nativelySupported := []string{"AminoDHT"} // 로컬에서 실행하는 시스템들
bootstrapPeers := config.GetBootstrapPeers(nativelySupported...)
delegatedEndpoints := config.GetDelegatedEndpoints(nativelySupported...)
dnsResolvers := config.GetDNSResolvers()
log.Printf("부트스트랩 피어: %v", bootstrapPeers)
log.Printf("위임된 엔드포인트: %v", delegatedEndpoints)
log.Printf("DNS 리졸버: %v", dnsResolvers)
// 애플리케이션 로직...
}
```

백그라운드 업데이트 패턴

패턴 1: 컨텍스트 기반 생명주기 (서비스 통합에 권장)

```
func runService(ctx context.Context) error {
```

```
client, err := autoconf.NewClient(
autoconf.WithCacheDir("/app/data/autoconf"),
autoconf.WithUserAgent("myapp/1.2.3"),
autoconf.WithRefreshInterval(12*time.Hour),
// 모니터링을 위한 사용자 정의 콜백
autoconf.WithOnNewVersion(func(oldVer, newVer int64, url string) {
log.Printf("새 설정 버전 %d 사용 가능", newVer)
}),
autoconf.WithOnRefreshError(func(err error) {
log.Printf("AutoConf 새로고침 실패: %v", err)
}),
)
if err != nil {
return err
}
// Start는 캐시를 준비하고 백그라운드 업데이터를 시작
// ctx가 취소되면 업데이터가 자동으로 중지됨
config, err := client.Start(ctx)
if err != nil {
return err
}
// 설정을 즉시 사용
bootstrapPeers := config.GetBootstrapPeers("AminoDHT")
```

```
log.Printf("부트스트랩 피어: %v", bootstrapPeers)

// 서비스 로직...

// ctx가 취소되면 업데이터가 자동으로 중지됨

return nil

}
```

패턴 2: 명시적 Stop() 메서드 (수동 제어에 권장)

```
func main() {

client, err := autoconf.NewClient(

autoconf.WithCacheDir("/app/data/autoconf"),

autoconf.WithUserAgent("myapp/1.2.3"),

)

if err != nil {

log.Fatal(err)

}

// Stop()을 사용할 것이므로 백그라운드 컨텍스트로 시작

config, err := client.Start(context.Background())

if err != nil {

log.Fatal(err)

}

// 깔끔한 종료 보장

defer client.Stop()

// 시그널 처리 설정
```

```
sigCh := make(chan os.Signal, 1)

signal.Notify(sigCh, syscall.SIGINT, syscall.SIGTERM)

// 애플리케이션 로직...

<-sigCh

// Stop()이 defer에 의해 호출되어 우아한 종료 보장

}
```

캐시 구조

캐시는 효율적인 관리를 위해 호스트명과 버전별로 구성됩니다:

```
$CACHE_DIR/

autoconf/

example.com/ # 호스트명 기반 디렉토리

autoconf-2025071801.json # 버전별 설정 파일

autoconf-2025071802.json # 신뢰성을 위해 여러 버전 유지

.etag # 조건부 요청을 위한 HTTP ETag

.last-modified # 캐시 검증을 위한 HTTP Last-Modified

.last-refresh # 마지막 새로고침 시도 타임스탬프

conf.ipfs-mainnet.org/

autoconf-2025072301.json

.etag

.last-modified
```

```
.last-refresh
```

캐시 특징

- **버전 기반 파일**: 각 AutoConfVersion이 자체 JSON 파일을 가짐
- **메타데이터 파일**: 효율적인 조건부 요청을 위한 HTTP 헤더 저장
- **자동 정리**: 오래된 버전이 자동으로 제거됨 (기본값: 3개 버전 유지)
- **호스트명 분리**: 여러 URL이 독립적으로 캐시됨

오류 처리

클라이언트는 우아한 폴백을 구현합니다:

1. 원격 URL에서 가져오기 시도
2. 원격 실패 시, 최신 캐시된 버전으로 폴백
3. 캐시가 없으면, 설정된 폴백 함수로 폴백

참고: `GetLatest()` 는 명시적 처리를 위해 오류를 반환하지만, `GetCached()` 와 `GetCachedOrRefresh()` 는 실패하지 않고 항상 사용 가능한 설정을 반환합니다 (필요시 폴백 사용).

모범 사례

1. 적절한 새로고침 간격 설정

```
// 프로덕션 환경: 24시간 (기본값)
autoconf.WithRefreshInterval(24 * time.Hour)

// 개발 환경: 더 짧은 간격
```

```
autoconf.WithRefreshInterval(1 * time.Hour)


// 테스트 환경: 매우 짧은 간격
autoconf.WithRefreshInterval(5 * time.Minute)
```

2. 적절한 캐시 디렉토리 설정

```
// 시스템 전체 캐시
autoconf.WithCacheDir("/var/cache/myapp/autoconf")


// 사용자별 캐시
autoconf.WithCacheDir(filepath.Join(os.Getenv("HOME"), ".cache/myapp/autoconf"))


// 애플리케이션 데이터 디렉토리
autoconf.WithCacheDir("/app/data/autoconf")
```

3. 모니터링과 로깅

```
client, err := autoconf.NewClient(
autoconf.WithOnNewVersion(func(oldVer, newVer  int64, url  string) {
// 메트릭 기록
metrics.RecordConfigUpdate(oldVer, newVer)
log.Infof("설정 업데이트: v%d -> v%d", oldVer, newVer)
```

```
}),
autoconf.WithOnRefreshError(func(err  error) {
// 오류 메트릭 기록
metrics.RecordConfigError()
// 알림 시스템에 전송
alerting.SendAlert("AutoConf 새로고침 실패", err)
}),
)
```

문제 해결

일반적인 문제들

1. 네트워크 연결 문제

```
// 타임아웃 증가
autoconf.WithTimeout(30 * time.Second)

// TLS 문제 (테스트 환경에서만)
autoconf.WithTLSInsecureSkipVerify(true)
```

2. 캐시 권한 문제

```
# 캐시 디렉토리 권한 확인
```

```
ls -la /path/to/cache/


# 권한 수정
chmod 755 /path/to/cache/
```

3. 디스크 공간 부족

```
// 캐시 크기 줄이기
autoconf.WithCacheSize(1) // 최소 1개 버전만 유지
```

디버깅

```
import "github.com/ipfs/go-log/v2"


// 로그 레벨 설정
logging.SetLogLevel("autoconf", "DEBUG")


// 또는 환경 변수로
// GOLOG_LOG_LEVEL=autoconf=DEBUG
```

이 문서는 Autoconf 모듈의 핵심 기능과 사용법을 다룹니다. 더 자세한 정보는 Boxo 라이브러리 개요와 Bootstrap 부트스트랩 문서를 참조하세요.